

Technical Architecture Document

Planetic Web Automated Website, Domain, Hosting, DNS, Billing & Client Management Platform

1. Document Purpose

This Technical Architecture Document defines the complete recommended architecture for building the Planetic Web platform.

The platform will allow customers to buy:

- A complete bespoke website package for £200
- Hosting packages
- Domain registration
- Domain + hosting bundles
- Optional website maintenance services in the future

The platform will automate:

- Customer account creation
- Online payment
- Domain registration
- Cloudflare DNS setup
- Namecheap WHM/cPanel hosting account creation
- Renewal tracking
- Renewal emails
- Failed payment handling
- Hosting suspension/reactivation
- Customer dashboard access
- Admin order and service management

This document is written for developers and Codex so the project can be built with a clear structure from the beginning.

2. Final Technology Decisions

2.1 Production Domain

The application will be deployed on:

`https://planeticweb.com`

2.2 Deployment Environment

The application will be deployed on **cPanel**.

The Laravel application should be stored outside the public web root where possible, and the domain document root should point to Laravel's `/public` directory.

Recommended structure:

```
/home/cpanelusername/planeticweb-app/  
/home/cpanelusername/planeticweb-app/public
```

The domain `planeticweb.com` should point to:

```
/home/cpanelusername/planeticweb-app/public
```

If the cPanel interface does not allow this directly, the developer must configure the domain/subdomain document root properly or use a safe public folder forwarding method. Laravel application files such as `.env`, `app`, `routes`, `database`, and `storage` must not be publicly accessible.

2.3 Hosting Provider for Customer Hosting Accounts

Use:

```
Namecheap Reseller Hosting with WHM/cPanel
```

The platform will use WHM API to create, suspend, unsuspend, and manage customer cPanel accounts.

2.4 Domain Registrar

Recommended primary registrar:

```
NameSilo API
```

Backup/alternative:

```
Namecheap Domain API
```

Porkbun should not be used as the primary registrar in the MVP unless full API support for domain registration, renewal, contact management, and nameserver updates is confirmed.

2.5 DNS Provider

Use:

Cloudflare DNS API

Cloudflare will manage customer domain DNS, SSL, CDN, and security settings.

2.6 Email Service

Use:

Namecheap cPanel Email with Jellyfish filtering

Important clarification:

Jellyfish is the filtering/spam management layer. The actual SMTP sending service will be cPanel Email.

Transactional emails will be sent from addresses such as:

no-reply@planeticweb.com
support@planeticweb.com
admin@planeticweb.com

2.7 Payment Provider

Use:

Stripe

Stripe will be used for:

- One-time payments
- Hosting subscriptions
- Domain renewals
- Payment method management
- Failed payment handling
- Invoice events
- Webhook-based automation

2.8 Application Framework

Use:

Laravel

Laravel is recommended because the platform needs authentication, dashboard pages, admin management, payment webhooks, queues, cron jobs, email notifications, database-driven workflows, and API integrations.

3. High-Level Architecture

3.1 System Overview

The system will work like this:

```
Customer visits planeticweb.com
↓
Customer chooses package
↓
Customer searches/selects domain
↓
Customer creates account
↓
Customer pays through Stripe
↓
Stripe sends verified webhook to Laravel
↓
Laravel creates order and starts provisioning jobs
↓
Domain is registered through NameSilo/Namecheap API
↓
Cloudflare zone is created
↓
Registrar nameservers are updated to Cloudflare nameservers
↓
WHM creates cPanel hosting account
↓
Cloudflare DNS records are created
↓
Customer receives welcome emails
↓
Customer manages domain, hosting, billing, and support from dashboard
```

3.2 Main Application Modules

The application will contain these core modules:

1. Public Website Module

2. Authentication Module
 3. Customer Dashboard Module
 4. Admin Dashboard Module
 5. Product and Pricing Module
 6. Cart and Checkout Module
 7. Stripe Billing Module
 8. Domain Registrar Module
 9. Namecheap WHM Hosting Module
 10. Cloudflare DNS Module
 11. Provisioning Engine
 12. Renewal Engine
 13. Email Notification Module
 14. Website Project Module
 15. Support Ticket Module
 16. Audit and Logging Module
-

4. Recommended Tech Stack

4.1 Backend

Technology

Laravel
PHP 8.2 or higher

Reasoning

Laravel is suitable because this project requires:

- User authentication
- Customer dashboard
- Admin dashboard
- Database models and relationships
- Stripe payment handling
- Stripe webhooks
- Background queue jobs
- Scheduled renewal checks
- Email notifications
- Secure environment configuration
- API service integrations
- cPanel-compatible deployment

Laravel gives a fast MVP path without overengineering the system.

4.2 Frontend

Technology

```
Laravel Blade  
Tailwind CSS  
Alpine.js  
Vite
```

Reasoning

For MVP, Laravel Blade is enough. The platform does not need a separate React/Next.js frontend at the start.

Blade + Tailwind + Alpine.js can handle:

- Marketing pages
- Pricing tables
- Checkout pages
- Login/register pages
- Customer dashboard
- Admin layout
- Forms
- Tables
- Modals
- Tabs
- Status badges

This keeps development faster and easier to deploy on cPanel.

4.3 Database

Technology

```
MySQL 8+
```

Reasoning

MySQL is recommended because:

- It works well with Laravel
 - It is supported by cPanel
 - The application has relational data
 - Users, orders, domains, invoices, hosting accounts, and subscriptions require clear relationships
 - It is easier to manage in a typical cPanel environment
-

4.4 Queue System

MVP

Use:

Laravel database queue

Reasoning

On cPanel, long-running queue workers may not always be available through Supervisor. For MVP, use the database queue and run queue processing using cron.

Queue jobs will handle:

- Domain registration
 - Cloudflare setup
 - WHM hosting account creation
 - DNS record creation
 - Sending emails
 - Renewal reminders
 - Failed payment processing
 - Suspension and unsuspension
-

4.5 Admin Panel

Recommended

Filament Admin Panel

Reasoning

Filament is recommended for Laravel because it allows fast creation of professional admin panels using Laravel models.

Admin will manage:

- Customers
- Orders
- Products
- Payments
- Invoices
- Domains
- Hosting accounts
- Cloudflare zones
- DNS records

- Provisioning failures
 - Website projects
 - Support tickets
-

4.6 Payment

Technology

Stripe Checkout
Stripe Billing
Stripe Webhooks

Reasoning

Stripe Checkout is secure and quick to implement. It reduces payment form complexity and allows the system to focus on service provisioning after verified payment.

The system must never provision services only because the frontend says payment was successful. Provisioning must happen only after a verified Stripe webhook.

4.7 Domain Registrar Integration

Recommended

NameSilo API

Backup

Namecheap Domain API

Reasoning

The registrar integration must support:

- Domain availability checks
- Domain registration
- Domain renewal
- Expiry date retrieval
- Nameserver updates
- Domain contact details
- Domain status sync

The code should use a registrar interface so the system can switch between NameSilo and Namecheap later.

4.8 Hosting Integration

Technology

Namecheap WHM API

Required WHM API Functions

The system should support:

- Create cPanel account
 - Suspend cPanel account
 - Unsuspend cPanel account
 - Terminate account if needed
 - Change hosting package
 - List accounts
 - Get account status
-

4.9 DNS Integration

Technology

Cloudflare API

Required Cloudflare Functions

The system should support:

- Create zone
 - Retrieve assigned nameservers
 - Create DNS records
 - Update DNS records
 - Delete DNS records
 - Set SSL mode
 - Enable Always Use HTTPS
 - Purge cache in future
 - Sync DNS status
-

4.10 Email

Technology

Namecheap cPanel Email SMTP
Jellyfish filtering

SMTP Mailboxes

Recommended mailboxes:

```
no-reply@planeticweb.com  
support@planeticweb.com  
admin@planeticweb.com  
billing@planeticweb.com
```

Important Note

cPanel Email is acceptable for MVP. If email delivery becomes poor later, transactional emails can be moved to a dedicated provider such as Mailgun, Resend, SMTP2GO, Amazon SES, or SendGrid without changing the core architecture.

5. Core Functional Architecture

5.1 Public Website

The public website will show:

- Home page
- £200 bespoke website package
- Hosting packages
- Domain search
- Pricing
- FAQs
- Contact page
- Privacy policy
- Terms of use
- Renewal policy

5.2 Customer Dashboard

Customers will be able to:

- View active services
- View domains
- View hosting packages
- View renewal dates
- View invoices
- View payment status
- Submit support tickets
- Complete website project intake form
- View website project status

5.3 Admin Dashboard

Admins will be able to:

- View customers
 - View orders
 - View payments
 - View invoices
 - View domains
 - View hosting accounts
 - View Cloudflare zones
 - View DNS records
 - Retry failed provisioning
 - Suspend/unsuspend hosting
 - Manage website projects
 - Manage support tickets
 - View audit logs
-

6. Project Folder Structure

```
planeticweb/
|
├── app/
│   ├── Actions/
│   │   ├── Checkout/
│   │   │   ├── CreateCheckoutSession.php
│   │   │   └── CompletePaidOrder.php
│   │   └── Domains/
│   │       ├── CheckDomainAvailability.php
│   │       ├── RegisterDomain.php
│   │       ├── RenewDomain.php
│   │       └── UpdateDomainNameservers.php
│   │   └── Hosting/
│   │       ├── CreateHostingAccount.php
│   │       ├── SuspendHostingAccount.php
│   │       ├── UnsuspendHostingAccount.php
│   │       └── SyncHostingAccount.php
│   │   └── Cloudflare/
│   │       ├── CreateCloudflareZone.php
│   │       ├── CreateDefaultDnsRecords.php
│   │       ├── EnableDefaultCloudflareSettings.php
│   │       └── SyncCloudflareStatus.php
│   └── Orders/
```

- └─ CreateOrderFromCart.php
 - └─ MarkOrderAsPaid.php
 - └─ MarkOrderAsFailed.php
 - └─ CancelOrder.php
 - └─ Renewals/
 - └─ GenerateRenewalReminder.php
 - └─ ProcessSuccessfulRenewal.php
 - └─ ProcessFailedRenewal.php
 - └─ WebsiteProjects/
 - └─ CreateWebsiteProject.php
 - └─ UpdateWebsiteProjectStatus.php
 - └─ RequestWebsiteContent.php
- └─ Console/
 - └─ Commands/
 - └─ CheckRenewalsCommand.php
 - └─ SyncDomainStatusesCommand.php
 - └─ SyncHostingStatusesCommand.php
 - └─ RetryFailedProvisioningCommand.php
- └─ Enums/
 - └─ DomainStatus.php
 - └─ HostingStatus.php
 - └─ InvoiceStatus.php
 - └─ OrderStatus.php
 - └─ PaymentStatus.php
 - └─ ProvisioningStatus.php
 - └─ SubscriptionStatus.php
 - └─ SupportTicketStatus.php
 - └─ WebsiteProjectStatus.php
- └─ Events/
 - └─ OrderPaid.php
 - └─ DomainRegistered.php
 - └─ HostingAccountCreated.php
 - └─ CloudflareZoneCreated.php
 - └─ PaymentFailed.php
 - └─ ServiceRenewalDue.php
- └─ Exceptions/
 - └─ RegistrarException.php
 - └─ WhmException.php
 - └─ CloudflareException.php
 - └─ ProvisioningException.php
- └─ Filament/
 - └─ Resources/
 - └─ UserResource.php

- └─ ProductResource.php
- └─ OrderResource.php
- └─ DomainResource.php
- └─ HostingAccountResource.php
- └─ SubscriptionResource.php
- └─ InvoiceResource.php
- └─ WebsiteProjectResource.php
- └─ SupportTicketResource.php
- └─ ProvisioningJobResource.php
- └─ Pages/
 - └─ Dashboard.php
 - └─ RevenueReport.php
 - └─ ProvisioningMonitor.php
- └─ Http/
 - └─ Controllers/
 - └─ Public/
 - └─ HomeController.php
 - └─ PricingController.php
 - └─ DomainSearchController.php
 - └─ ContactController.php
 - └─ Customer/
 - └─ DashboardController.php
 - └─ DomainController.php
 - └─ HostingController.php
 - └─ BillingController.php
 - └─ WebsiteProjectController.php
 - └─ SupportTicketController.php
 - └─ CheckoutController.php
 - └─ Webhook/
 - └─ StripeWebhookController.php
 - └─ Middleware/
 - └─ EnsureUserIsAdmin.php
 - └─ EnsureEmailVerified.php
 - └─ VerifyStripeWebhook.php
 - └─ Requests/
 - └─ CheckoutRequest.php
 - └─ DomainSearchRequest.php
 - └─ WebsiteProjectIntakeRequest.php
 - └─ SupportTicketRequest.php
 - └─ UpdateBillingDetailsRequest.php
- └─ Jobs/
 - └─ Provisioning/
 - └─ ProvisionOrderJob.php

- └─ RegisterDomainJob.php
- └─ CreateCloudflareZoneJob.php
- └─ UpdateRegistrarNameserversJob.php
- └─ CreateWhmHostingAccountJob.php
- └─ CreateCloudflareDnsRecordsJob.php
- └─ SendProvisioningCompletedEmailJob.php
- ─ Billing/
 - └─ HandleStripeInvoicePaidJob.php
 - └─ HandleStripePaymentFailedJob.php
 - └─ SyncStripeSubscriptionJob.php
- ─ Renewals/
 - └─ SendRenewalReminderJob.php
 - └─ SuspendOverdueHostingJob.php
 - └─ UnsuspendPaidHostingJob.php
- ─ Notifications/
 - └─ SendWelcomeEmailJob.php
 - └─ SendOrderConfirmationEmailJob.php
 - └─ SendSupportTicketEmailJob.php
- ─ Mail/
 - └─ OrderConfirmationMail.php
 - └─ PaymentFailedMail.php
 - └─ HostingWelcomeMail.php
 - └─ DomainRegisteredMail.php
 - └─ RenewalReminderMail.php
 - └─ SupportTicketUpdatedMail.php
- ─ Models/
 - └─ User.php
 - └─ Role.php
 - └─ Product.php
 - └─ ProductPrice.php
 - └─ HostingPackage.php
 - └─ Cart.php
 - └─ CartItem.php
 - └─ Order.php
 - └─ OrderItem.php
 - └─ Payment.php
 - └─ Invoice.php
 - └─ Subscription.php
 - └─ Domain.php
 - └─ DomainContact.php
 - └─ HostingAccount.php
 - └─ CloudflareZone.php
 - └─ DnsRecord.php
 - └─ ProvisioningJob.php
 - └─ WebsiteProject.php

- WebsiteProjectAsset.php
 - SupportTicket.php
 - SupportTicketMessage.php
 - NotificationLog.php
 - AuditLog.php
 - Policies/
 - DomainPolicy.php
 - HostingAccountPolicy.php
 - InvoicePolicy.php
 - SupportTicketPolicy.php
 - WebsiteProjectPolicy.php
 - Services/
 - Billing/
 - StripeService.php
 - InvoiceService.php
 - Registrar/
 - RegistrarInterface.php
 - NameSiloRegistrar.php
 - NamecheapRegistrar.php
 - RegistrarResponseParser.php
 - Hosting/
 - WhmService.php
 - CpanelPackageMapper.php
 - DNS/
 - CloudflareService.php
 - DefaultDnsRecordBuilder.php
 - Provisioning/
 - ProvisioningOrchestrator.php
 - ProvisioningLogger.php
 - ProvisioningRetryService.php
 - Renewals/
 - RenewalService.php
 - SuspensionService.php
- bootstrap/
- config/
 - app.php
 - auth.php
 - billing.php
 - cloudflare.php
 - domain.php
 - hosting.php

```
|
|   ├── mail.php
|   ├── queue.php
|   ├── services.php
|   ├── stripe.php
|   └── whm.php
|
|   ├── database/
|   |   ├── migrations/
|   |   ├── seeders/
|   |   |   ├── DatabaseSeeder.php
|   |   |   ├── RoleSeeder.php
|   |   |   ├── ProductSeeder.php
|   |   |   └── HostingPackageSeeder.php
|   |   └── factories/
|   |
|   ├── public/
|   |   ├── index.php
|   |   ├── assets/
|   |   └── images/
|   |
|   ├── resources/
|   |   ├── css/
|   |   |   └── app.css
|   |   ├── js/
|   |   |   └── app.js
|   |   └── views/
|   |       ├── layouts/
|   |       |   ├── app.blade.php
|   |       |   ├── public.blade.php
|   |       |   ├── customer.blade.php
|   |       |   └── admin.blade.php
|   |       ├── public/
|   |       |   ├── home.blade.php
|   |       |   ├── pricing.blade.php
|   |       |   ├── website-package.blade.php
|   |       |   ├── hosting.blade.php
|   |       |   ├── domain-search.blade.php
|   |       |   ├── contact.blade.php
|   |       |   ├── privacy-policy.blade.php
|   |       |   ├── terms.blade.php
|   |       |   └── renewal-policy.blade.php
|   |       ├── auth/
|   |       |   ├── login.blade.php
|   |       |   ├── register.blade.php
|   |       |   └── forgot-password.blade.php
```



```

├── checkout/
│   ├── cart.blade.php
│   ├── checkout.blade.php
│   ├── success.blade.php
│   └── failed.blade.php
├── customer/
│   ├── dashboard.blade.php
│   ├── domains/
│   ├── hosting/
│   ├── billing/
│   ├── website-projects/
│   └── support/
├── emails/
│   ├── order-confirmation.blade.php
│   ├── hosting-welcome.blade.php
│   ├── domain-registered.blade.php
│   ├── payment-failed.blade.php
│   ├── renewal-reminder.blade.php
│   └── support-ticket-updated.blade.php
├── components/
│   ├── pricing-card.blade.php
│   ├── service-status-badge.blade.php
│   ├── invoice-status-badge.blade.php
│   └── dns-record-table.blade.php
├── routes/
│   ├── web.php
│   ├── auth.php
│   ├── customer.php
│   ├── admin.php
│   ├── api.php
│   └── console.php
├── storage/
├── tests/
│   ├── Feature/
│   │   ├── CheckoutTest.php
│   │   ├── DomainSearchTest.php
│   │   ├── StripeWebhookTest.php
│   │   ├── ProvisioningTest.php
│   │   └── CustomerDashboardTest.php
│   └── Unit/
│       ├── NameSiloRegistrarTest.php
│       ├── WhmServiceTest.php
│       └── CloudflareServiceTest.php

```

```
|      └─ RenewalServiceTest.php
|
|─ .env.example
|─ artisan
|─ composer.json
|─ package.json
|─ vite.config.js
└─ README.md
```

7. Routes

7.1 Public Routes

GET	/	Home page
GET	/website-package	£200 bespoke website package
GET	/hosting	Hosting packages
GET	/domains	Domain search page
POST	/domains/search	Domain availability search
GET	/contact	Contact page
GET	/privacy-policy	Privacy policy
GET	/terms	Terms of use
GET	/renewal-policy	Renewal policy

7.2 Authentication Routes

```
GET /login
POST /login
GET /register
POST /register
POST /logout
GET /forgot-password
POST /forgot-password
```

7.3 Checkout Routes

```
GET /cart
POST /cart/add
POST /cart/remove
GET /checkout
POST /checkout/start
GET /checkout/success
GET /checkout/cancel
```

7.4 Customer Dashboard Routes

```
GET /dashboard
GET /dashboard/domains
GET /dashboard/domains/{domain}
GET /dashboard/domains/{domain}/dns
POST /dashboard/domains/{domain}/dns
GET /dashboard/hosting
GET /dashboard/hosting/{hostingAccount}
GET /dashboard/billing
GET /dashboard/invoices/{invoice}
GET /dashboard/website-projects
GET /dashboard/website-projects/{project}
POST /dashboard/website-projects/{project}/intake
GET /dashboard/support
POST /dashboard/support
GET /dashboard/support/{ticket}
POST /dashboard/support/{ticket}/reply
```

7.5 Webhook Routes

```
POST /webhooks/stripe
```

7.6 Admin Routes

```
GET /admin
```

Admin routes will mainly be handled through Filament.

8. Database Schema

8.1 users

Stores customer and admin accounts.

id	bigint primary key
name	varchar
email	varchar unique
email_verified_at	timestamp nullable
password	varchar
phone	varchar nullable
company_name	varchar nullable
billing_address_line_1	varchar nullable

billing_address_line_2	varchar nullable
billing_city	varchar nullable
billing_state	varchar nullable
billing_postcode	varchar nullable
billing_country	varchar nullable
stripe_customer_id	varchar nullable
is_admin	boolean default false
status	varchar default active
remember_token	varchar nullable
created_at	timestamp
updated_at	timestamp

Relationship:

- One user can have many orders
- One user can have many domains
- One user can have many hosting accounts
- One user can have many invoices
- One user can have many subscriptions
- One user can have many website projects
- One user can have many support tickets

8.2 roles

Stores admin/customer/staff roles.

id	bigint primary key
name	varchar unique
label	varchar
created_at	timestamp
updated_at	timestamp

Example roles:

```
admin
support
developer
customer
```

8.3 role_user

Connects users to roles.

id	bigint primary key
user_id	foreign key users.id
role_id	foreign key roles.id
created_at	timestamp
updated_at	timestamp

Relationship:

- One user can have multiple roles
- One role can belong to many users

8.4 products

Stores sellable products.

id	bigint primary key
name	varchar
slug	varchar unique
type	varchar
description	text nullable
is_active	boolean default true
sort_order	integer default 0
created_at	timestamp
updated_at	timestamp

Product types:

```
website_package
hosting
domain
maintenance
addon
```

Examples:

- Complete Bespoke Website
- Starter Hosting
- Business Hosting
- Pro Hosting
- Domain Registration
- Website Maintenance

8.5 product_prices

Stores product pricing.

id	bigint primary key
product_id	foreign key products.id
billing_cycle	varchar
currency	varchar default GBP
amount	decimal(10,2)
setup_fee	decimal(10,2) default 0
stripe_price_id	varchar nullable
is_active	boolean default true
created_at	timestamp
updated_at	timestamp

Billing cycles:

```
one_time
monthly
yearly
```

Relationship:

- One product can have many prices

8.6 hosting_packages

Maps Planetic Web hosting packages to WHM packages.

id	bigint primary key
product_id	foreign key products.id
name	varchar
whm_package_name	varchar
disk_limit_mb	integer nullable
bandwidth_limit_mb	integer nullable
email_accounts_limit	integer nullable
database_limit	integer nullable
domain_limit	integer nullable
is_active	boolean default true
created_at	timestamp
updated_at	timestamp

Example mapping:

Starter Hosting → planetic_starter
Business Hosting → planetic_business
Pro Hosting → planetic_pro
Agency Hosting → planetic_agency

8.7 carts

Stores customer cart before checkout.

id	bigint primary key
user_id	foreign key users.id nullable
session_id	varchar nullable
currency	varchar default GBP
subtotal	decimal(10,2) default 0
discount_total	decimal(10,2) default 0
tax_total	decimal(10,2) default 0
total	decimal(10,2) default 0
status	varchar default active
created_at	timestamp
updated_at	timestamp

8.8 cart_items

Stores individual cart items.

id	bigint primary key
cart_id	foreign key carts.id
product_id	foreign key products.id
product_price_id	foreign key product_prices.id nullable
item_type	varchar
name	varchar
domain_name	varchar nullable
quantity	integer default 1
unit_price	decimal(10,2)
total	decimal(10,2)
metadata	json nullable
created_at	timestamp
updated_at	timestamp

Item types:

website_package
hosting

```
domain_registration
domain_renewal
addon
maintenance
```

8.9 orders

Stores confirmed customer orders.

```
id                bigint primary key
user_id           foreign key users.id
order_number      varchar unique
status            varchar
payment_status    varchar
currency          varchar default GBP
subtotal          decimal(10,2)
discount_total    decimal(10,2) default 0
tax_total         decimal(10,2) default 0
total            decimal(10,2)
stripe_checkout_session_id varchar nullable
stripe_payment_intent_id  varchar nullable
stripe_subscription_id    varchar nullable
paid_at             timestamp nullable
cancelled_at        timestamp nullable
created_at          timestamp
updated_at          timestamp
```

Order statuses:

```
pending
paid
provisioning
completed
failed
cancelled
refunded
manual_review
```

8.10 order_items

Stores products inside an order.

```
id                bigint primary key
order_id          foreign key orders.id
```


product_id	foreign key products.id nullable
product_price_id	foreign key product_prices.id nullable
item_type	varchar
name	varchar
domain_name	varchar nullable
quantity	integer default 1
unit_price	decimal(10,2)
total	decimal(10,2)
metadata	json nullable
created_at	timestamp
updated_at	timestamp

8.11 payments

Stores payment attempts and successful payments.

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id nullable
invoice_id	foreign key invoices.id nullable
provider	varchar default stripe
provider_payment_id	varchar nullable
provider_customer_id	varchar nullable
amount	decimal(10,2)
currency	varchar default GBP
status	varchar
failure_reason	text nullable
paid_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Payment statuses:

```

pending
succeeded
failed
refunded
partially_refunded
cancelled

```

8.12 invoices

Stores invoices for orders, subscriptions, and renewals.

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id nullable
subscription_id	foreign key subscriptions.id nullable
invoice_number	varchar unique
stripe_invoice_id	varchar nullable
currency	varchar default GBP
subtotal	decimal(10,2)
tax_total	decimal(10,2) default 0
total	decimal(10,2)
amount_paid	decimal(10,2) default 0
amount_due	decimal(10,2) default 0
status	varchar
due_date	date nullable
paid_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Invoice statuses:

```

draft
open
paid
void
uncollectible
failed
refunded

```

8.13 subscriptions

Stores recurring services.

id	bigint primary key
user_id	foreign key users.id
product_id	foreign key products.id
domain_id	foreign key domains.id nullable
hosting_account_id	foreign key hosting_accounts.id nullable
stripe_subscription_id	varchar nullable
status	varchar
billing_cycle	varchar
currency	varchar default GBP
amount	decimal(10,2)
current_period_start	date nullable
current_period_end	date nullable
next_renewal_date	date nullable
cancel_at_period_end	boolean default false

cancelled_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Subscription statuses:

```
active
past_due
cancelled
unpaid
paused
trialing
expired
```

8.14 domains

Stores customer domains.

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id nullable
domain_name	varchar unique
sld	varchar
tld	varchar
registrar	varchar
registrar_domain_id	varchar nullable
registrar_order_id	varchar nullable
status	varchar
registration_date	date nullable
expiry_date	date nullable
auto_renew	boolean default true
whois_privacy	boolean default true
registrar_lock	boolean default true
cloudflare_zone_id	foreign key cloudflare_zones.id nullable
nameservers	json nullable
last_synced_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Domain statuses:

```
pending
available_checked
registration_pending
active
expired
```

```
renewal_due
renewal_failed
suspended
cancelled
failed
manual_review
```

Relationship:

- Domain belongs to one user
- Domain can have one Cloudflare zone
- Domain can have many DNS records
- Domain can have one hosting account
- Domain can have one subscription

8.15 domain_contacts

Stores contact details required for domain registration.

```
id                bigint primary key
domain_id         foreign key domains.id
contact_type      varchar
first_name        varchar
last_name         varchar
company_name      varchar nullable
email             varchar
phone             varchar
address_line_1    varchar
address_line_2    varchar nullable
city              varchar
state             varchar nullable
postcode          varchar
country           varchar
created_at        timestamp
updated_at        timestamp
```

Contact types:

```
registrant
admin
technical
billing
```

8.16 hosting_accounts

Stores cPanel hosting accounts created through WHM.

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id nullable
domain_id	foreign key domains.id nullable
hosting_package_id	foreign key hosting_packages.id
domain_name	varchar
whm_username	varchar unique
whm_account_id	varchar nullable
server_hostname	varchar nullable
server_ip	varchar
cpanel_url	varchar nullable
status	varchar
disk_limit_mb	integer nullable
bandwidth_limit_mb	integer nullable
created_on_whm_at	timestamp nullable
suspended_at	timestamp nullable
suspension_reason	text nullable
renewal_date	date nullable
last_synced_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Hosting statuses:

```
pending
creating
active
suspended
terminated
failed
manual_review
```

Relationship:

- Hosting account belongs to one user
- Hosting account belongs to one hosting package
- Hosting account usually belongs to one domain

8.17 cloudflare_zones

Stores Cloudflare zone information.

id	bigint primary key
user_id	foreign key users.id
domain_id	foreign key domains.id
zone_id	varchar unique
zone_name	varchar
status	varchar
name_servers	json nullable
ssl_mode	varchar nullable
always_use_https	boolean default true
created_on_cloudflare_at	timestamp nullable
last_synced_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Cloudflare zone statuses:

```

pending
active
initializing
moved
deleted
failed
manual_review

```

Relationship:

- Cloudflare zone belongs to one domain
- Cloudflare zone has many DNS records

8.18 dns_records

Stores DNS records managed through Cloudflare.

id	bigint primary key
user_id	foreign key users.id
domain_id	foreign key domains.id
cloudflare_zone_id	foreign key cloudflare_zones.id
cloudflare_record_id	varchar nullable
type	varchar
name	varchar
content	text
ttl	integer nullable
proxied	boolean default false
priority	integer nullable
status	varchar default active
metadata	json nullable

created_at	timestamp
updated_at	timestamp

DNS record types:

```
A
AAAA
CNAME
MX
TXT
NS
SRV
CAA
```

8.19 provisioning_jobs

Tracks automation jobs.

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id
job_type	varchar
status	varchar
attempts	integer default 0
max_attempts	integer default 3
started_at	timestamp nullable
completed_at	timestamp nullable
failed_at	timestamp nullable
error_message	text nullable
request_payload	json nullable
response_payload	json nullable
created_at	timestamp
updated_at	timestamp

Job types:

```
register_domain
create_cloudflare_zone
update_nameservers
create_hosting_account
create_dns_records
send_welcome_email
```

Job statuses:

```
pending
running
completed
failed
retrying
cancelled
manual_review
```

8.20 website_projects

Stores bespoke website project information.

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id
domain_id	foreign key domains.id nullable
hosting_account_id	foreign key hosting_accounts.id nullable
project_number	varchar unique
status	varchar
business_name	varchar nullable
business_description	text nullable
industry	varchar nullable
pages_required	json nullable
brand_colours	varchar nullable
reference_websites	json nullable
special_requirements	text nullable
content_received	boolean default false
logo_received	boolean default false
target_launch_date	date nullable
launched_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Website project statuses:

```
order_received
information_required
content_received
design_in_progress
review_required
revisions_in_progress
approved
launched
completed
```



```
on_hold
cancelled
```

8.21 website_project_assets

Stores uploaded project files.

```
id                bigint primary key
website_project_id foreign key website_projects.id
user_id           foreign key users.id
file_type         varchar
original_filename varchar
stored_path       varchar
mime_type         varchar nullable
file_size         integer nullable
created_at        timestamp
updated_at        timestamp
```

File types:

```
logo
image
content_document
brand_guideline
other
```

8.22 support_tickets

Stores customer support tickets.

```
id                bigint primary key
user_id           foreign key users.id
assigned_admin_id foreign key users.id nullable
ticket_number     varchar unique
subject          varchar
category         varchar nullable
priority         varchar default normal
status           varchar default open
created_at        timestamp
updated_at        timestamp
closed_at         timestamp nullable
```

Ticket statuses:

```
open
pending
waiting_customer
resolved
closed
```

Priorities:

```
low
normal
high
urgent
```

8.23 support_ticket_messages

Stores messages inside support tickets.

```
id                bigint primary key
support_ticket_id foreign key support_tickets.id
user_id           foreign key users.id
message           text
is_internal_note  boolean default false
created_at        timestamp
updated_at        timestamp
```

8.24 notification_logs

Tracks emails and notifications.

```
id                bigint primary key
user_id           foreign key users.id nullable
type              varchar
channel           varchar
recipient         varchar
subject           varchar nullable
status            varchar
sent_at           timestamp nullable
failed_at         timestamp nullable
error_message     text nullable
metadata          json nullable
created_at        timestamp
updated_at        timestamp
```

8.25 audit_logs

Stores system and admin activity.

id	bigint primary key
user_id	foreign key users.id nullable
action	varchar
entity_type	varchar nullable
entity_id	bigint nullable
description	text nullable
old_values	json nullable
new_values	json nullable
ip_address	varchar nullable
user_agent	text nullable
created_at	timestamp

8.26 coupons

Stores discount codes.

id	bigint primary key
code	varchar unique
description	text nullable
discount_type	varchar
discount_value	decimal(10,2)
max_uses	integer nullable
used_count	integer default 0
starts_at	timestamp nullable
expires_at	timestamp nullable
is_active	boolean default true
created_at	timestamp
updated_at	timestamp

Discount types:

fixed
percentage

8.27 coupon_redemptions

Tracks coupon use.

id	bigint primary key
coupon_id	foreign key coupons.id
user_id	foreign key users.id
order_id	foreign key orders.id
discount_amount	decimal(10,2)
created_at	timestamp
updated_at	timestamp

9. Entity Relationships in Plain English

9.1 User

A user can be a customer or admin.

A user can have:

- Many orders
- Many payments
- Many invoices
- Many subscriptions
- Many domains
- Many hosting accounts
- Many website projects
- Many support tickets
- Many uploaded website assets

9.2 Order

An order belongs to one user.

An order can contain:

- Website package
- Domain registration
- Hosting package
- Add-ons
- Maintenance package

An order can create:

- A domain record
- A hosting account
- A Cloudflare zone
- A website project
- One or more provisioning jobs

9.3 Domain

A domain belongs to one user.

A domain can have:

- One Cloudflare zone
- Many DNS records
- One hosting account
- One subscription
- Multiple contact records

9.4 Hosting Account

A hosting account belongs to:

- One user
- One hosting package
- Usually one domain

The hosting account is created inside Namecheap WHM.

9.5 Cloudflare Zone

A Cloudflare zone belongs to:

- One user
- One domain

A Cloudflare zone can have many DNS records.

9.6 Subscription

A subscription belongs to:

- One user
- One product

It may be connected to:

- A domain
- A hosting account
- A maintenance plan

9.7 Website Project

A website project belongs to:

- One user
- One order

It may also be linked to:

- One domain
- One hosting account

A website project can have many uploaded files.

10. Provisioning Workflow

10.1 Full Website Package Provisioning

When a customer buys the £200 website package:

1. Customer selects package
2. Customer searches/selects domain
3. Customer creates account
4. Customer pays through Stripe
5. Stripe webhook confirms payment
6. System creates order
7. System creates website project
8. System registers domain
9. System creates Cloudflare zone
10. System updates registrar nameservers
11. System creates WHM hosting account
12. System creates DNS records
13. System sends welcome emails
14. Customer completes website intake form
15. Admin starts website design/development

10.2 Hosting-Only Provisioning

1. Customer selects hosting package
2. Customer enters existing domain or buys new domain
3. Customer pays
4. Stripe webhook confirms payment
5. System creates WHM hosting account
6. System creates Cloudflare zone if required
7. System creates DNS records
8. Customer receives hosting welcome email

10.3 Domain-Only Provisioning

1. Customer searches domain
2. Customer selects available domain
3. Customer pays

4. Stripe webhook confirms payment
5. System registers domain
6. System creates Cloudflare zone
7. System updates nameservers
8. System stores expiry date
9. Customer receives domain registration email

11. Provisioning Engine Design

Provisioning must use Laravel queues because it involves external APIs.

11.1 Provisioning Job Chain

```
ProvisionOrderJob
  → RegisterDomainJob
  → CreateCloudflareZoneJob
  → UpdateRegistrarNameserversJob
  → CreateWhmHostingAccountJob
  → CreateCloudflareDnsRecordsJob
  → SendProvisioningCompletedEmailJob
```

11.2 Important Rules

Every provisioning step must:

- Be logged
- Be retryable
- Be idempotent
- Store request/response where safe
- Avoid duplicate domain registration
- Avoid duplicate WHM account creation
- Avoid duplicate Cloudflare zone creation
- Notify admin if failed

11.3 Idempotency

Before creating anything externally, check:

- Does the domain already exist in database?
- Is the domain already registered?
- Does a Cloudflare zone already exist?
- Does a WHM account already exist?
- Do DNS records already exist?

This avoids duplicate services if a job retries.

12. API Integration Design

12.1 Registrar Interface

Create a generic registrar interface.

```
interface RegistrarInterface
{
    public function checkAvailability(string $domain): array;

    public function registerDomain(array $data): array;

    public function renewDomain(string $domain, int $years = 1): array;

    public function getDomainInfo(string $domain): array;

    public function updateNameservers(string $domain, array $nameservers):
array;
}
```

Implement:

```
NameSiloRegistrar.php
NamecheapRegistrar.php
```

This keeps the platform flexible.

12.2 WHM Service

Create:

```
app/Services/Hosting/WhmService.php
```

Required methods:

```
createAccount()
suspendAccount()
unsuspendAccount()
terminateAccount()
listAccounts()
getAccountInfo()
changePackage()
```


WHM API credentials must only be stored in `.env`.

12.3 Cloudflare Service

Create:

```
app/Services/DNS/CloudflareService.php
```

Required methods:

```
createZone()  
getZone()  
listZones()  
createDnsRecord()  
updateDnsRecord()  
deleteDnsRecord()  
setSslMode()  
enableAlwaysUseHttps()  
purgeCache()
```

12.4 Stripe Service

Create:

```
app/Services/Billing/StripeService.php
```

Handle these Stripe events:

```
checkout.session.completed  
payment_intent.succeeded  
payment_intent.payment_failed  
invoice.paid  
invoice.payment_failed  
customer.subscription.created  
customer.subscription.updated  
customer.subscription.deleted
```

Important:

- Always verify webhook signature
- Do not provision from frontend success URL
- Store Stripe customer ID
- Store Stripe payment intent ID

- Store Stripe subscription ID
- Store Stripe invoice ID

13. Cloudflare DNS Setup

For each customer domain, create these records by default:

A	@	customer hosting server IP	proxied
CNAME	www	customerdomain.com	proxied
A	mail	customer hosting server IP	DNS only
MX	@	mail.customerdomain.com	DNS only
TXT	@	SPF record	DNS only
TXT	_dmarc	DMARC record	DNS only

Do not proxy these records:

```
mail
webmail
cpanel
whm
ftp
autodiscover
MX
SPF
DKIM
DMARC
```

Only proxy:

```
@
www
```

14. Email Configuration

14.1 Main Platform Email

Use cPanel Email for:

```
no-reply@planeticweb.com
support@planeticweb.com
```

```
admin@planeticweb.com
billing@planeticweb.com
```

14.2 SMTP Settings

Production `.env` should use:

```
MAIL_MAILER=smtp
MAIL_HOST=mail.planeticweb.com
MAIL_PORT=465
MAIL_USERNAME=no-reply@planeticweb.com
MAIL_PASSWORD=YOUR_EMAIL_PASSWORD
MAIL_ENCRYPTION=ssl
MAIL_FROM_ADDRESS=no-reply@planeticweb.com
MAIL_FROM_NAME="Planetic Web"
```

Alternative TLS setup:

```
MAIL_PORT=587
MAIL_ENCRYPTION=tls
```

14.3 Cloudflare DNS for Email

Because Cloudflare will manage DNS, email records must be DNS-only.

Required records:

A	mail	cPanel server IP	DNS only
A	webmail	cPanel server IP	DNS only
MX	@	mail.planeticweb.com	DNS only
TXT	@	SPF record	DNS only
TXT	_dmarc	DMARC record	DNS only
TXT	DKIM	DKIM from cPanel	DNS only

14.4 Email Deliverability Warning

cPanel Email is acceptable for MVP. However, payment emails and renewal reminders are critical. If delivery becomes unreliable, move transactional emails to a specialist SMTP provider later.

15. Environment Variables

Use this `.env.example`:

```
APP_NAME="Planetic Web"
APP_ENV=production
APP_KEY=
APP_DEBUG=false
APP_URL=https://planeticweb.com
APP_TIMEZONE=Europe/London

LOG_CHANNEL=stack
LOG_LEVEL=debug

DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=YOUR_CPANEL_DATABASE_NAME
DB_USERNAME=YOUR_CPANEL_DATABASE_USER
DB_PASSWORD=YOUR_DATABASE_PASSWORD

SESSION_DRIVER=database
CACHE_STORE=database
QUEUE_CONNECTION=database

MAIL_MAILER=smtp
MAIL_HOST=mail.planeticweb.com
MAIL_PORT=465
MAIL_USERNAME=no-reply@planeticweb.com
MAIL_PASSWORD=YOUR_EMAIL_PASSWORD
MAIL_ENCRYPTION=ssl
MAIL_FROM_ADDRESS=no-reply@planeticweb.com
MAIL_FROM_NAME="Planetic Web"

STRIPE_PUBLIC_KEY=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
STRIPE_CURRENCY=gbp

BILLING_GRACE_PERIOD_DAYS=7
RENEWAL_REMINDER_DAYS_BEFORE=30,14,7,3,1

DOMAIN_REGISTRAR=namesilo

NAMESILO_API_KEY=
NAMESILO_API_ENDPOINT=https://www.namesilo.com/api
NAMESILO_SANDBOX=false

NAMECHEAP_API_USER=
NAMECHEAP_API_KEY=
NAMECHEAP_USERNAME=
NAMECHEAP_CLIENT_IP=
NAMECHEAP_API_ENDPOINT=https://api.namecheap.com/xml.response
NAMECHEAP_SANDBOX=false
```

```
WHM_HOST=
WHM_PORT=2087
WHM_USERNAME=
WHM_API_TOKEN=
WHM_VERIFY_SSL=true
WHM_DEFAULT_PACKAGE=planetic_starter
WHM_SERVER_IP=
WHM_CPANEL_LOGIN_URL=

CLOUDFLARE_API_TOKEN=
CLOUDFLARE_ACCOUNT_ID=
CLOUDFLARE_DEFAULT_SSL_MODE=full
CLOUDFLARE_ALWAYS_USE_HTTPS=true
CLOUDFLARE_PROXIED_WEBSITE_RECORDS=true

DEFAULT_DNS_TTL=1
DEFAULT_MAIL_SERVER_HOSTNAME=mail
DEFAULT_DMARC_POLICY=none

ADMIN_EMAIL=admin@planeticweb.com
SUPPORT_EMAIL=support@planeticweb.com
BILLING_EMAIL=billing@planeticweb.com

FILESYSTEM_DISK=local
```

16. Configuration Files

16.1 config/whm.php

```
return [
    'host' => env('WHM_HOST'),
    'port' => env('WHM_PORT', 2087),
    'username' => env('WHM_USERNAME'),
    'token' => env('WHM_API_TOKEN'),
    'verify_ssl' => env('WHM_VERIFY_SSL', true),
    'default_package' => env('WHM_DEFAULT_PACKAGE', 'planetic_starter'),
    'server_ip' => env('WHM_SERVER_IP'),
    'cpanel_login_url' => env('WHM_CPANEL_LOGIN_URL'),
];
```

16.2 config/cloudflare.php

```
return [
    'api_token' => env('CLOUDFLARE_API_TOKEN'),
    'account_id' => env('CLOUDFLARE_ACCOUNT_ID'),
```

```

'default_ssl_mode' => env('CLOUDFLARE_DEFAULT_SSL_MODE', 'full'),
'always_use_https' => env('CLOUDFLARE_ALWAYS_USE_HTTPS', true),
'proxied_website_records' => env('CLOUDFLARE_PROXIED_WEBSITE_RECORDS',
true),
];

```

16.3 config/domain.php

```

return [
    'default_registrar' => env('DOMAIN_REGISTRAR', 'namesilo'),

    'namesilo' => [
        'api_key' => env('NAMESILO_API_KEY'),
        'endpoint' => env('NAMESILO_API_ENDPOINT', 'https://www.namesilo.com/
api'),
        'sandbox' => env('NAMESILO_SANDBOX', false),
    ],

    'namecheap' => [
        'api_user' => env('NAMECHEAP_API_USER'),
        'api_key' => env('NAMECHEAP_API_KEY'),
        'username' => env('NAMECHEAP_USERNAME'),
        'client_ip' => env('NAMECHEAP_CLIENT_IP'),
        'endpoint' => env('NAMECHEAP_API_ENDPOINT'),
        'sandbox' => env('NAMECHEAP_SANDBOX', false),
    ],
];

```

16.4 config/billing.php

```

return [
    'currency' => env('STRIPE_CURRENCY', 'gbp'),
    'grace_period_days' => env('BILLING_GRACE_PERIOD_DAYS', 7),
    'renewal_reminder_days_before' => explode(',',
env('RENEWAL_REMINDER_DAYS_BEFORE', '30,14,7,3,1')),
];

```

17. cPanel Deployment Notes

17.1 Recommended Folder Setup

```

/home/cpanelusername/planeticweb-app
/home/cpanelusername/planeticweb-app/public

```

The domain `planeticweb.com` should point to:

```
/home/cpanelusername/planeticweb-app/public
```

17.2 Required Deployment Commands

After uploading the project:

```
composer install --no-dev --optimize-autoloader
php artisan key:generate
php artisan migrate --force
php artisan db:seed --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan storage:link
```

If Node is available:

```
npm install
npm run build
```

If Node is not available on cPanel, build assets locally and upload the compiled assets.

18. Cron Jobs on cPanel

18.1 Laravel Scheduler

Add this cron job:

```
* * * * * cd /home/cpanelusername/planeticweb-app && /usr/local/bin/php
artisan schedule:run >> /dev/null 2>&1
```

18.2 Queue Worker Using Cron

Because this is cPanel, use a cron-based queue worker:

```
* * * * * cd /home/cpanelusername/planeticweb-app && /usr/local/bin/php
artisan queue:work --stop-when-empty --tries=3 --timeout=120 >> /dev/null
2>&1
```

This is safer than trying to keep a permanent queue worker running on cPanel.

19. Scheduled Tasks

In Laravel scheduler, configure:

```
Schedule::command('renewals:check')->dailyAt('09:00');  
Schedule::command('domains:sync')->dailyAt('02:00');  
Schedule::command('hosting:sync')->dailyAt('03:00');  
Schedule::command('provisioning:retry-failed')->hourly();
```

Renewal reminders should be sent:

```
30 days before renewal  
14 days before renewal  
7 days before renewal  
3 days before renewal  
1 day before renewal  
On renewal day
```

20. Security Requirements

20.1 Authentication

Use Laravel authentication with:

- Secure password hashing
- Email verification
- Password reset
- Session protection
- CSRF protection

20.2 Authorization

Customers must only access their own:

- Domains
- Hosting accounts
- Invoices
- Payments
- Subscriptions
- Website projects
- Support tickets

Admins can access all records.

20.3 API Key Security

Never expose these keys in frontend JavaScript:

```
STRIPE_SECRET_KEY  
STRIPE_WEBHOOK_SECRET  
WHM_API_TOKEN  
CLOUDFLARE_API_TOKEN  
NAMESILO_API_KEY  
NAMECHEAP_API_KEY  
MAIL_PASSWORD
```

20.4 Stripe Webhook Security

The Stripe webhook must:

- Verify webhook signature
- Reject invalid signatures
- Store event IDs
- Avoid duplicate processing
- Trigger provisioning only after verified payment

20.5 Rate Limiting

Apply rate limits to:

- Login
- Registration
- Domain search
- Checkout start
- Support ticket creation
- Stripe webhook endpoint

20.6 Audit Logging

Log these actions:

- Admin login
 - Admin changes customer details
 - Admin retries provisioning
 - Admin suspends hosting
 - Admin unsuspends hosting
 - Admin cancels service
 - Admin edits renewal date
 - Admin changes payment/invoice status
-

21. Renewal Automation

21.1 Hosting Renewal

Flow:

1. System detects upcoming renewal
2. Sends reminder emails
3. Stripe attempts payment
4. If payment succeeds, renewal date is extended
5. If payment fails, customer is notified
6. Grace period starts
7. If unpaid after grace period, hosting is suspended
8. If customer pays later, hosting is unsuspended

21.2 Domain Renewal

Flow:

1. System detects upcoming domain renewal
2. Customer receives reminders
3. Stripe attempts payment before registrar renewal date
4. If payment succeeds, domain is renewed through registrar API
5. If payment fails, admin is alerted
6. Domain is marked renewal_failed/manual_review

Important:

Domain renewal should be handled carefully because once a registrar renews the domain, Planeti Web may be charged even if customer payment fails.

22. Error Handling

22.1 Domain Registration Fails

System should:

- Mark domain as failed
- Mark order as manual review
- Notify admin
- Show customer friendly message
- Allow admin to retry or offer alternative domain

22.2 WHM Account Creation Fails

System should:

- Mark hosting account as failed
- Log WHM response
- Notify admin
- Allow admin retry
- Prevent duplicate cPanel account creation

22.3 Cloudflare Setup Fails

System should:

- Mark DNS setup as failed
- Notify admin
- Allow manual retry
- Continue order as pending/manual review

22.4 Email Sending Fails

System should:

- Log email failure
- Mark notification as failed
- Allow admin to resend
- Not block core provisioning unless email is business-critical

22.5 Payment Fails

System should:

- Keep order unpaid
 - Do not provision services
 - Show retry payment option
 - Send failed payment email if possible
-

23. MVP Scope

23.1 MVP Must Include

Customer side:

- Public website
- £200 website package page
- Hosting packages page
- Domain search

- Customer registration/login
- Stripe checkout
- Order confirmation
- Customer dashboard
- Domain details
- Hosting details
- Invoice/payment status
- Renewal date display
- Website intake form
- Support ticket form

Admin side:

- Admin login
- Customer management
- Order management
- Payment status
- Domain status
- Hosting status
- Website project status
- Support ticket management
- Provisioning logs
- Manual retry button
- Manual suspend/unsuspend button

Automation:

- Stripe payment confirmation
- Domain registration
- WHM hosting account creation
- Cloudflare zone creation
- Basic DNS record creation
- Welcome emails
- Renewal reminder emails
- Failed payment emails

23.2 MVP Should Not Include

- Full DNS editor
 - Domain transfers
 - Advanced cPanel replacement
 - White-label reseller accounts
 - AI website builder
 - Advanced reporting
 - Affiliate system
 - Live chat
 - Mobile app
-

24. MVP Build Sequence

Phase 1: Foundation

1. Install Laravel.
2. Configure database.
3. Add authentication.
4. Add public pages.
5. Add products and prices.
6. Add Filament admin panel.
7. Add customer dashboard shell.

Phase 2: Checkout

1. Add cart system.
2. Add package selection.
3. Add domain search.
4. Add Stripe Checkout.
5. Add Stripe webhook.
6. Store orders and payments.

Phase 3: Provisioning

1. Build registrar service.
2. Build WHM service.
3. Build Cloudflare service.
4. Build provisioning jobs.
5. Add provisioning logs.
6. Add admin retry function.

Phase 4: Dashboard

1. Show domains.
2. Show hosting accounts.
3. Show invoices.
4. Show renewal dates.
5. Add website intake form.
6. Add support tickets.

Phase 5: Renewals

1. Add renewal reminder scheduler.
 2. Add failed payment handling.
 3. Add hosting suspension rules.
 4. Add reactivation after payment.
 5. Add admin renewal alerts.
-

25. Testing Strategy

25.1 Unit Tests

Test:

- Domain availability parser
- Registrar response parser
- WHM username generation
- Cloudflare DNS record builder
- Renewal date calculation
- Price calculation
- Coupon calculation

25.2 Feature Tests

Test:

- Registration
- Login
- Domain search
- Cart add/remove
- Checkout start
- Stripe webhook processing
- Order creation
- Provisioning job creation
- Customer dashboard permissions
- Support ticket creation

25.3 Integration Tests

Test with sandbox/test environments where possible:

- Stripe checkout
- Stripe webhook
- Registrar availability check
- Cloudflare zone creation
- WHM test account creation

Do not repeatedly test live domain registration because it can cost money.

26. Production Launch Checklist

Before launch:

```
APP_DEBUG=false  
APP_URL=https://planeticweb.com
```

```
HTTPS enabled
Database created
Migrations completed
Admin user created
Products seeded
Hosting packages mapped to WHM packages
Stripe live keys added
Stripe webhook configured
WHM API token added
Cloudflare API token added
Registrar API key added
SMTP mailbox created
Email DNS records configured
Cloudflare DNS configured
Queue cron running
Scheduler cron running
Test order completed
Test payment completed
Test provisioning completed
Test email received
Test failed payment flow completed
Database backup enabled
```

27. Important Business Rules

27.1 Free Domain and Hosting

The £200 website package must say:

```
Free domain and hosting for the first year.
Renewal applies after the first year.
```

Do not say:

```
Free domain and hosting forever.
```

27.2 Provisioning Rule

Services must be provisioned only after verified Stripe payment webhook.

Do not provision after frontend redirect alone.

27.3 DNS Rule

Only website records should be proxied through Cloudflare.

Email and service records must be DNS-only.

27.4 Renewal Rule

Domain renewal payments should be collected before renewing the domain at registrar level.

27.5 Failed Automation Rule

Every failed automation step must be visible in admin panel and retryable.

28. Developer Rules

1. Do not call third-party APIs directly from controllers.
 2. Use service classes for Stripe, registrar, WHM, and Cloudflare.
 3. Use queues for all provisioning tasks.
 4. Log every provisioning step.
 5. Make provisioning jobs idempotent.
 6. Never expose secret keys to frontend.
 7. Never trust frontend payment success.
 8. Provision only after verified Stripe webhook.
 9. Keep registrar integration replaceable.
 10. Keep customer data isolated by user ID.
 11. Use clear status fields for all services.
 12. Add manual admin retry for failed automation.
 13. Use cron jobs for queues and scheduler on cPanel.
 14. Keep Laravel app files outside public web root.
 15. Keep email records DNS-only in Cloudflare.
-

29. Final Architecture Summary

The final architecture for Planetic Web is:

```
Laravel monolithic application
+ MySQL database
+ Laravel Blade / Tailwind CSS frontend
+ Filament admin panel
+ Stripe Checkout and Billing
+ Stripe webhooks
+ NameSilo API as primary registrar
+ Namecheap Domain API as backup registrar
+ Namecheap WHM API for hosting automation
```


- + Cloudflare API for DNS/security
- + Namecheap cPanel Email SMTP
- + Jellyfish email filtering
- + Laravel database queues
- + Laravel scheduler through cPanel cron
- + cPanel deployment on planeticweb.com

This architecture is realistic for MVP, suitable for cPanel deployment, and flexible enough to scale later.

The most important principle is to keep the system modular:

- Registrar can be changed later.
- Hosting provider can be changed later.
- Email provider can be changed later.
- Frontend can be upgraded later.
- Queue system can move from database to Redis later.
- Admin panel can be expanded later.

Start with a clean Laravel monolith, build strong service classes, use queues for automation, and make every external API step logged and retryable.